

CS4432: Database Systems II

B-Tree Index Structure

Updates

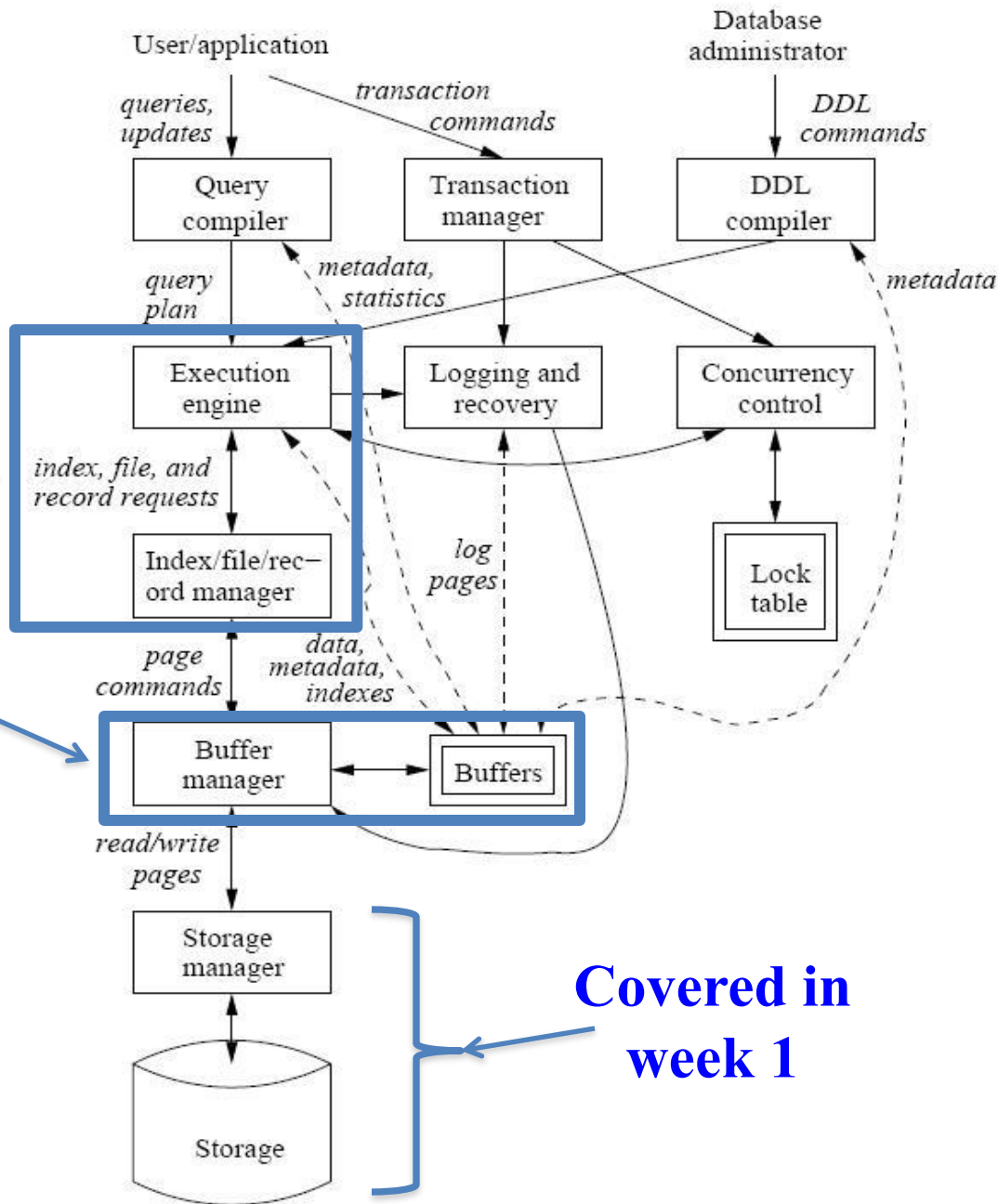
- Today: **Quiz 3**
- Grades:
Quiz1 – posted
HW1 – tonight
- HW 2 is out
Deadline: **Thu Apr 9**

**Started in
week 3**

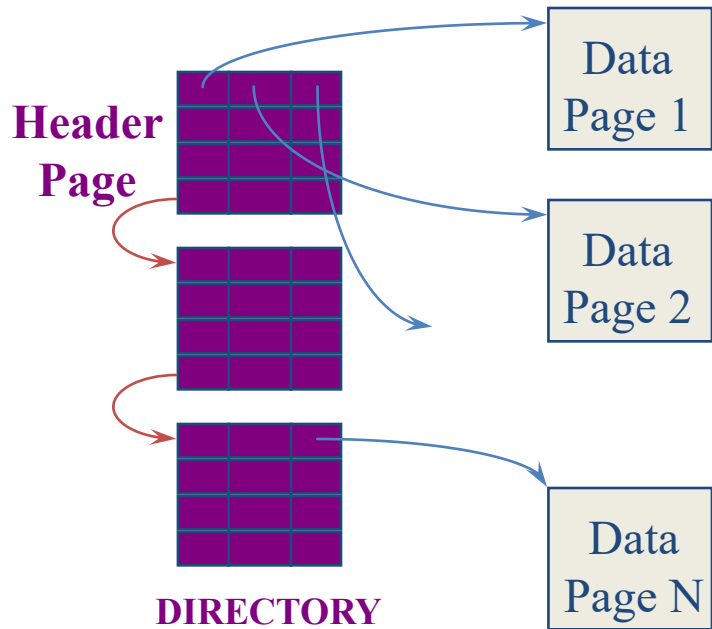
**Covered
in week 2**

**Also covered
in week 3:
sorting**

**Covered in
week 1**



Locating Records: Table Scans



- Open the files of relation R
- Access each data page
- Access each record in each page
- Check the condition



What is the
I/O cost?

N



What is the least amount of
memory needed for Table Scan?

**Only 1 memory
block**

Sparse vs. Dense Indexes

- **Dense**
 - More space
 - Must use for unsorted files (secondary indexes)
 - Can tell if record does not exist without checking the data file
- **Sparse**
 - Less space
 - Only for sorted files or higher-level indexes
 - Cannot tell if record does not exist without checking data file

Primary Index

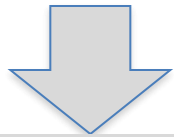
Clustered
column

- **Primary Index is:**

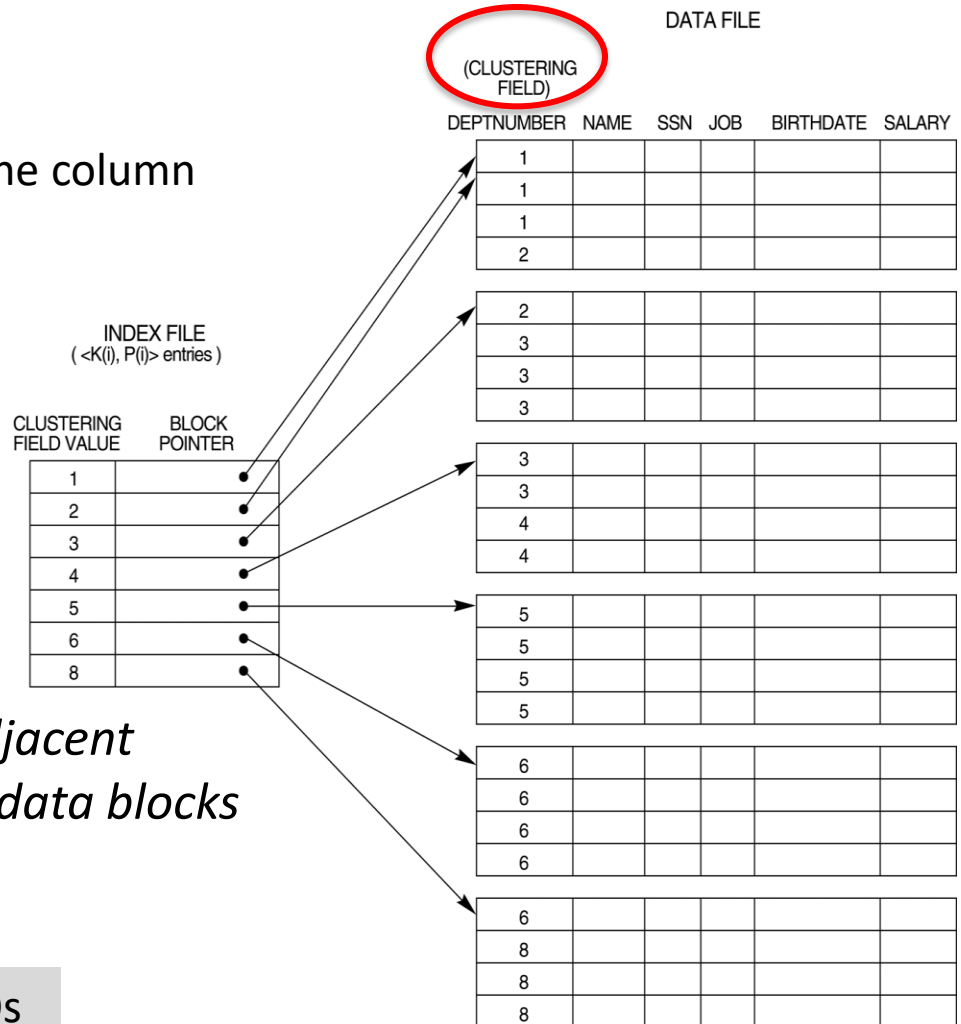
- An index on the ordering column (the column on which the data file is sorted)
- Can be dense or sparse
- Can be one-level or multi-level

Big Advantage

- *Records having the same key (or adjacent keys) are in the same (or adjacent) data blocks*



Leads to sequential I/Os



Secondary Indexes



Can we build a sparse index on un-ordered column??

No. We must have an index entry for each data record.

- The file may be ordered on another column, say **Name**.
 - An index on the **Name** column is primary index (Can be sparse or dense)
 - An index on any other column, say **ID**, is called **secondary index** (has to be dense)

Un-clustered

column



30	
50	

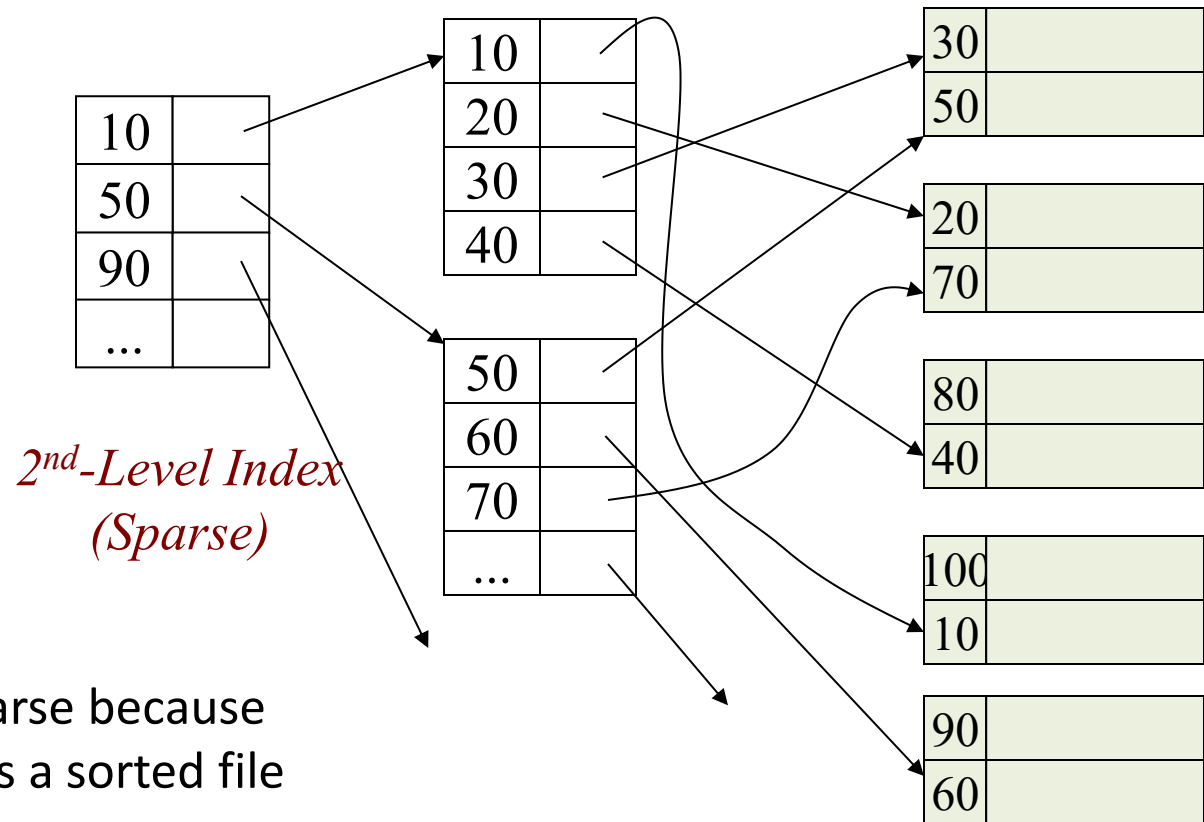
20	
70	

80	
40	

100	
10	

90	
60	

Multi-Level Secondary Indexes



- 2nd level can be sparse because the 1st level index is a sorted file

- Lowest level is dense
- Other levels are sparse

Example

Assume a relation consists of 100,000 data records

- An index is built on an unsorted column (i.e., a dense index).
- Each disk block can store 100 data records
- Each disk block can store 200 index entries

How many disk blocks are required for the second-level index?

$$\frac{100,000 \text{ records}}{100 \text{ records per block}} = 1,000 \text{ data blocks} \quad \frac{100,000 \text{ entries}}{200 \text{ entries per block}} = 500 \text{ index blocks}$$

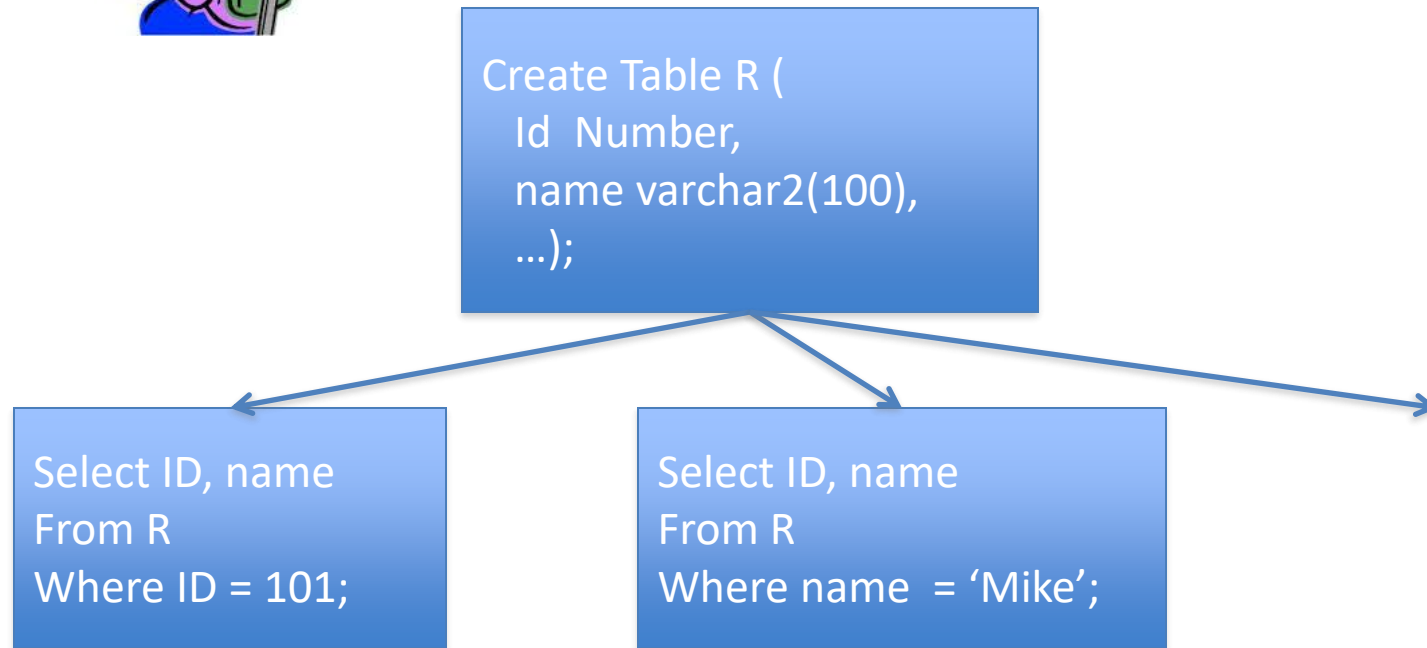
First must be dense -> will hold 500 index blocks.

Second can hold 200 entries, so

$$\left\lceil \frac{500 \text{ entries}}{200 \text{ entries per block}} \right\rceil = 3 \text{ index blocks:}$$



End-User View



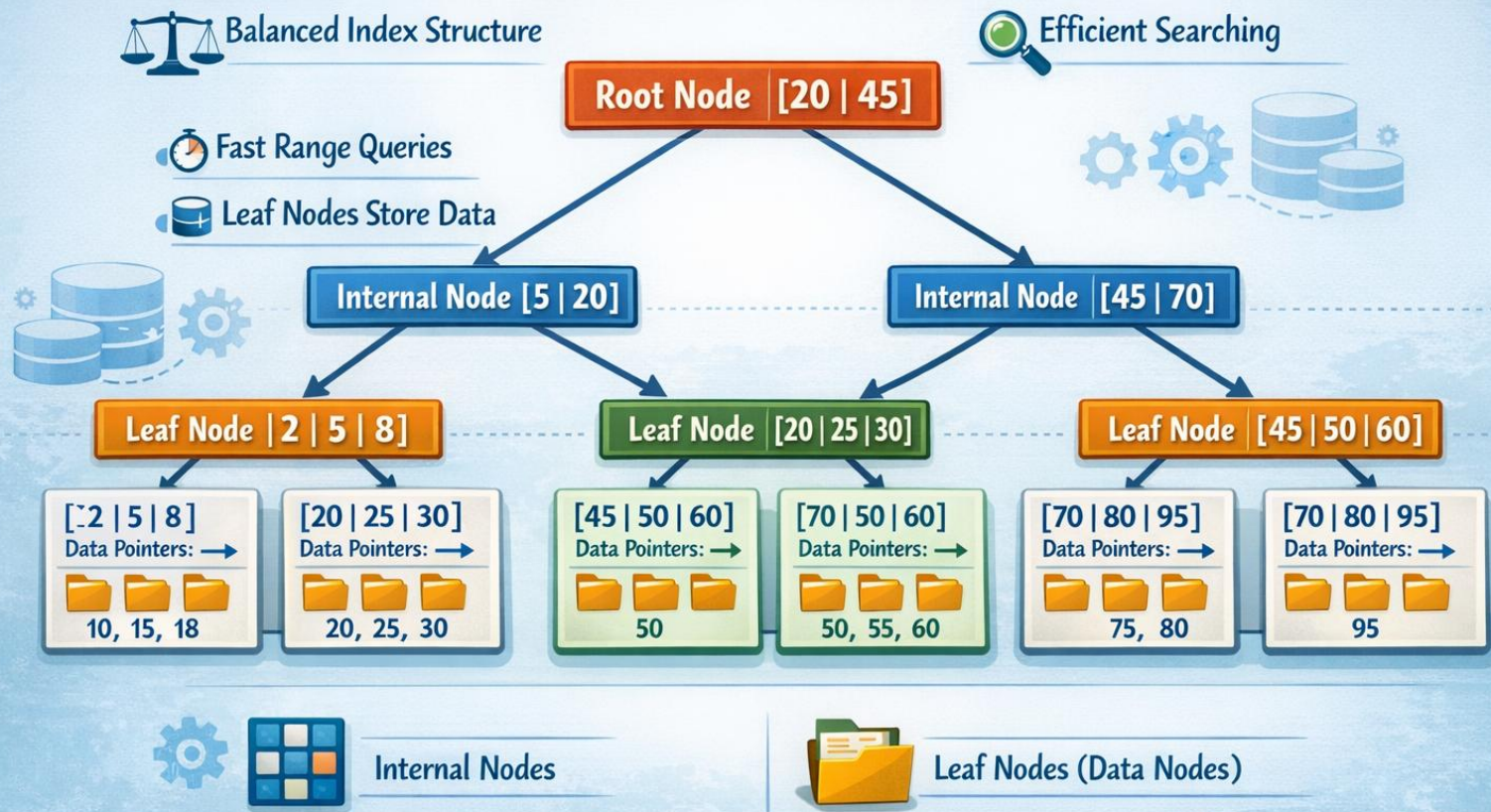
To speedup queries → We create indexes

```
> Create Index R_ID on R (ID);
```

```
> Create Index R_Name on R (name);
```

Today we will discuss B-trees

B+ Trees for Database Indexes



Learning objectives

- Explain why **B-trees** are **good data structures** to store DB **indexes**
- Determine if n-ary tree is a **valid** B-tree
- Compute **min, avg, max # indexed records** in B-tree
- Implement **equality** and **range** search in a B-tree

B-tree Family

There is a specific data structure called a B-tree.

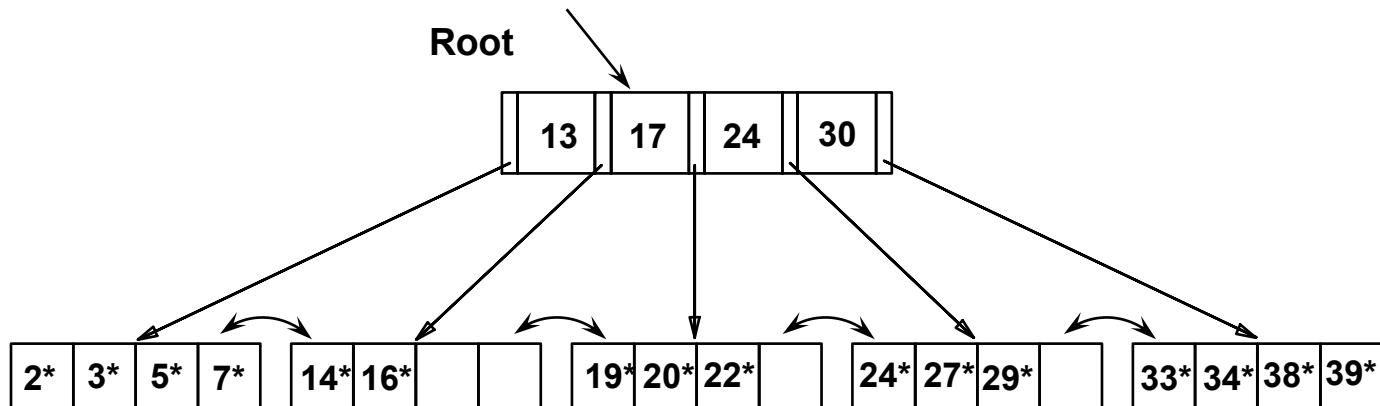
People also use the term to generally refer to a class of balanced data tree structures:

- B-tree (1971)
- B+Tree (1973)
- B*Tree (1977?)
- B^{link}-Tree (1981)

What Is B-Tree

- A **disk-based** multi-level balanced tree index structure

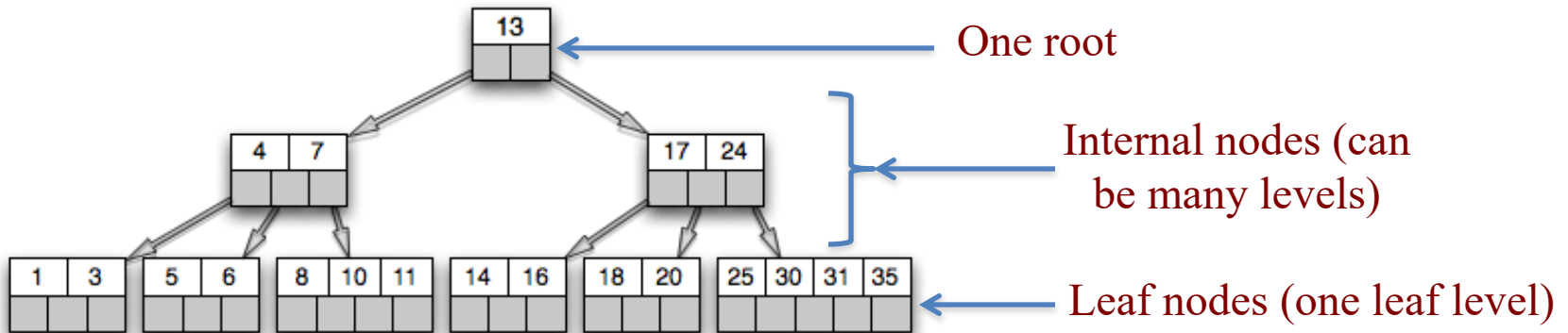
- Each node in the tree is a disk page → Has to fit in one disk page
- The pointers are locations on disk either block ids or record ids



What Is B-Tree

- A disk-based multi-level balanced tree index structure

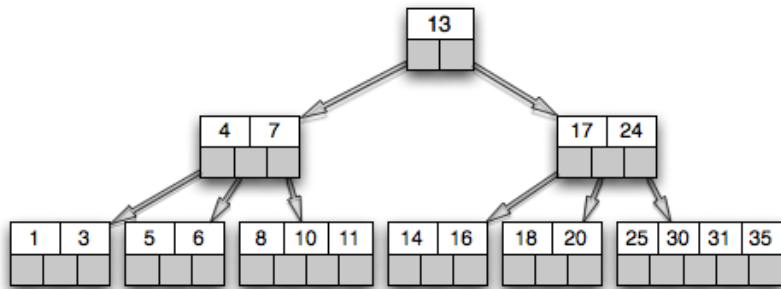
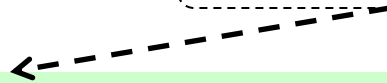
- Has one root (can be the only level)
- The height of the tree grows and shrinks dynamically as needed
- We always (in search, insert, delete) start from the root and move down one level at a time



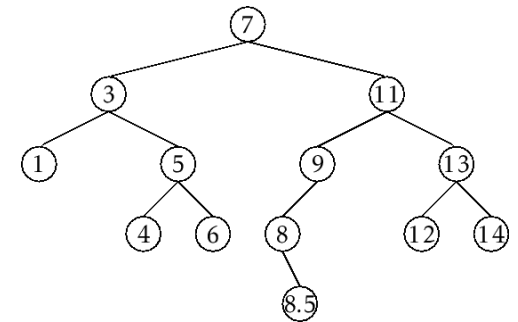
What Is B-Tree

- A disk-based multi-level **balanced tree** index structure

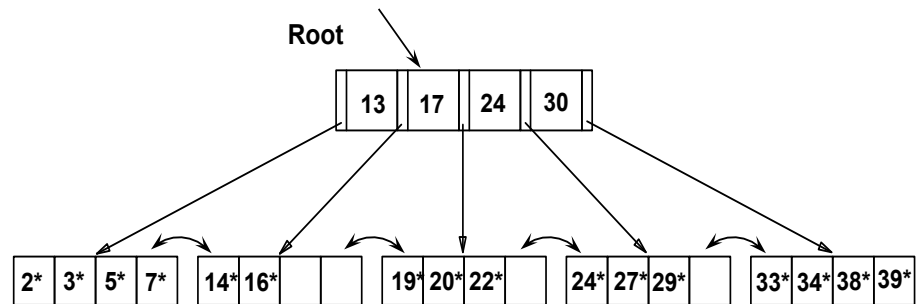
- All leaf nodes are at the same height



Height 3



Imbalanced tree



Height 2

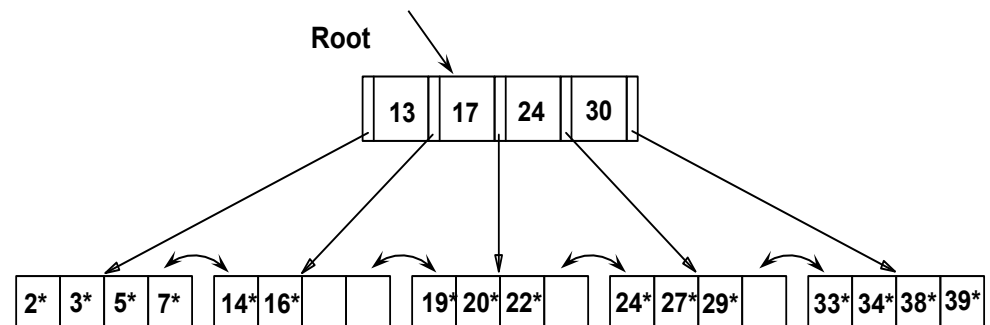
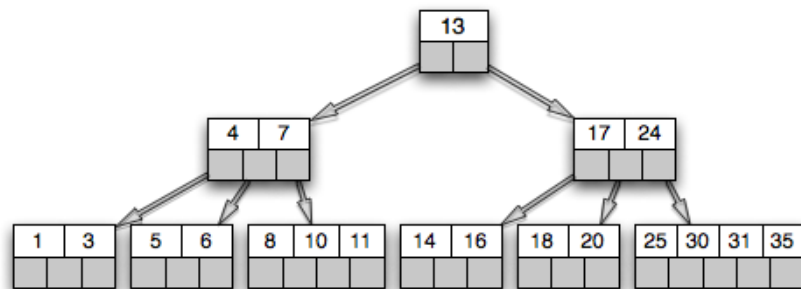
B+ Tree

- A B+ Tree is a self-balancing, ordered tree data structure that allows **searches, sequential access, insertions, and deletions in $O(\log n)$** .
 - Generalization of a binary search tree, since a node can have more than two children.
 - Optimized for systems that read and write large blocks of data.

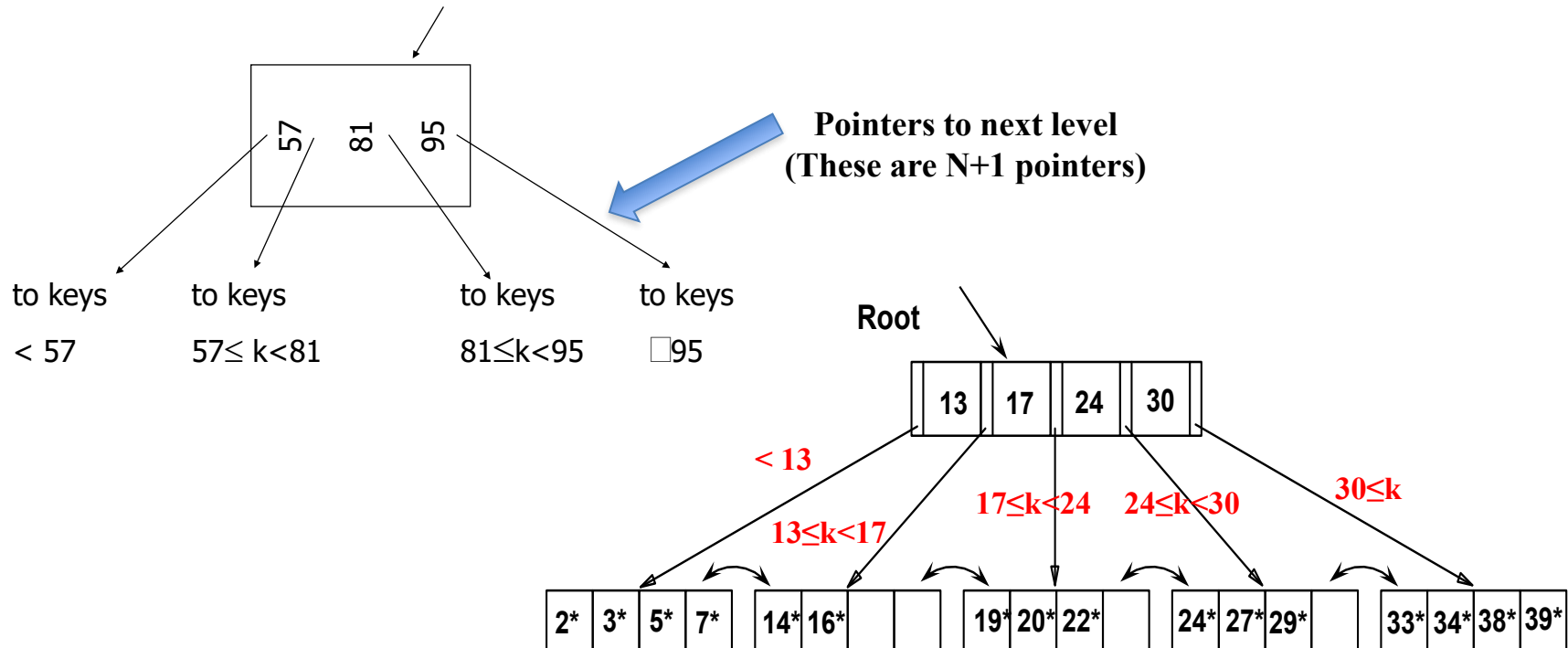
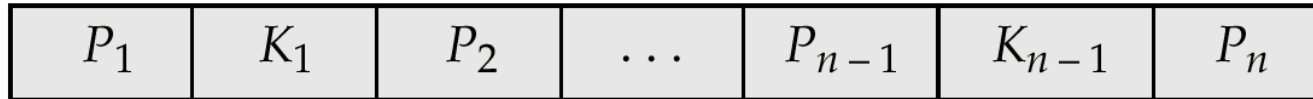
B-Tree Properties

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

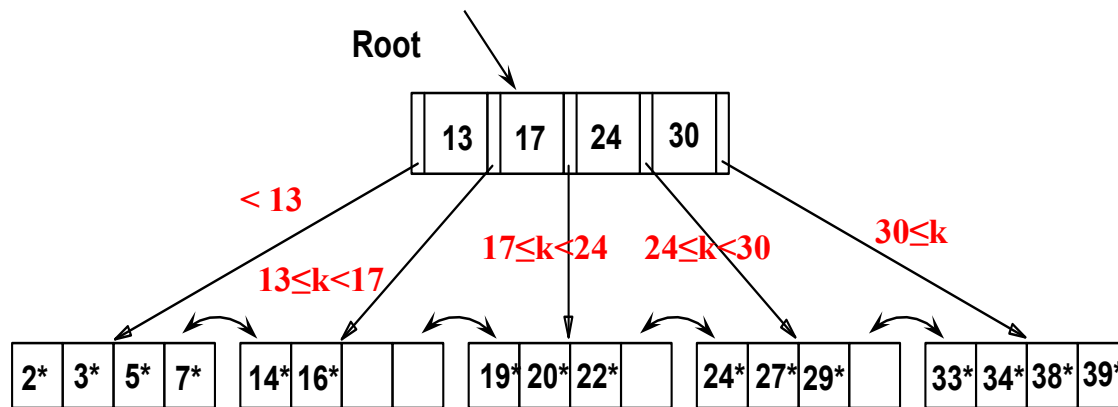
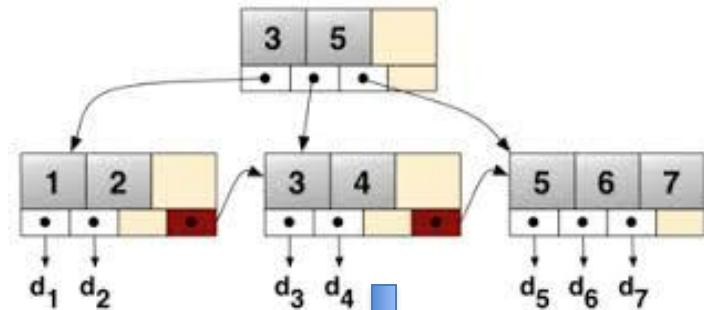
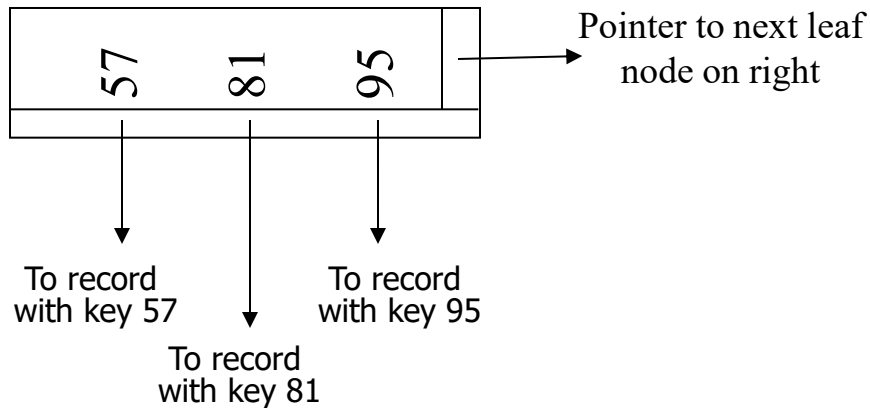
- Each node (which is disk block) holds N keys & N+1 pointers
- Keys in any node are sorted (left to right)
- Keys in any level are also sorted (left to right)
- Each node is at least 50% full (Minimum 50% occupancy)
 - Except the **Root** can have as minimum (1 key & 2 pointers)
- No empty spaces in the middle of a node
- Number of pointers in any node = number of keys + 1
 - The rest are Null



B⁺ Tree **Non-Leaf Node** Structure (root or inner/internal node)



B⁺ Tree Leaf Node Structure



These are the leaf nodes

LEAF NODE VALUES

Approach #1: Record IDs

- A pointer to the location of the tuple to which the index entry corresponds.

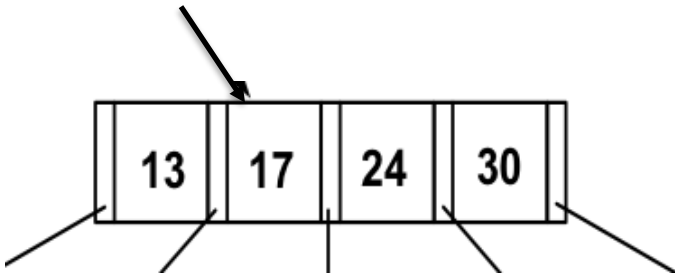
Approach #2: Tuple Data

- The leaf nodes store the actual contents of the tuple.
- Secondary indexes must store the Record ID as their values.

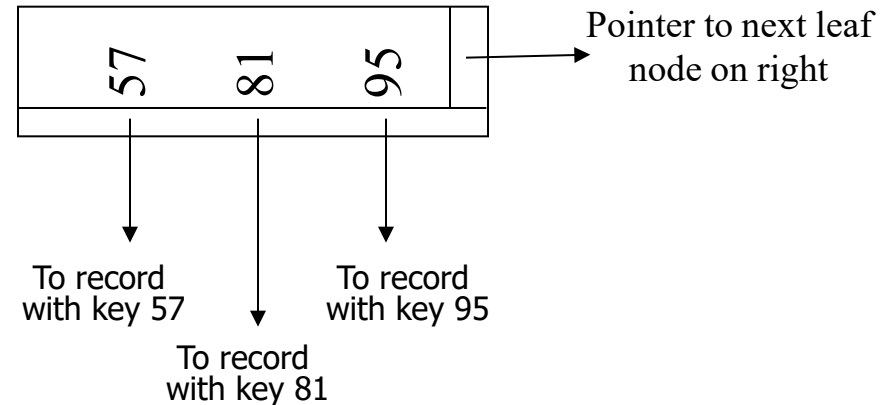
An Alternative Visualization

Arrange the pointers like this

Root or Internal Node



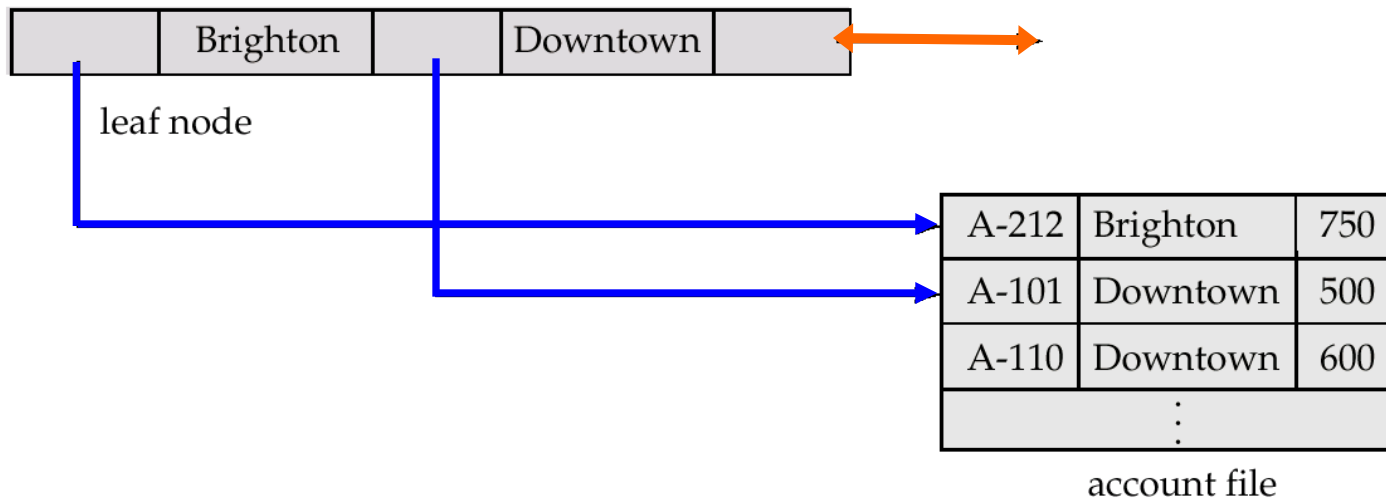
Leaf Node



Leaf Nodes in B-Tree

Properties of a leaf node:

- For each entry i in a leaf node:
 - pointer P_i points to a file record with search-key value K_i .
- P_n points to next leaf node in search-key order



Good Utilization

- B-tree nodes should not be too empty
- Each node is at least 50% full (Minimum 50% occupancy)
 - Except the **Root** can have as minimum (1 key & 2 pointers)

- Use at least
 - Non-leaf: $\lceil (n+1)/2 \rceil$ pointers
 - Leaf: $\lfloor (n+1)/2 \rfloor$ pointers to data

E.g.: $n = 3$
non-leaf: 2 ptr (1 val)
leaf: 2 val

E.g.: $n = 4$
non-leaf: 3 ptr (2 val)
leaf: 2 val

B-TREE VS. B+TREE

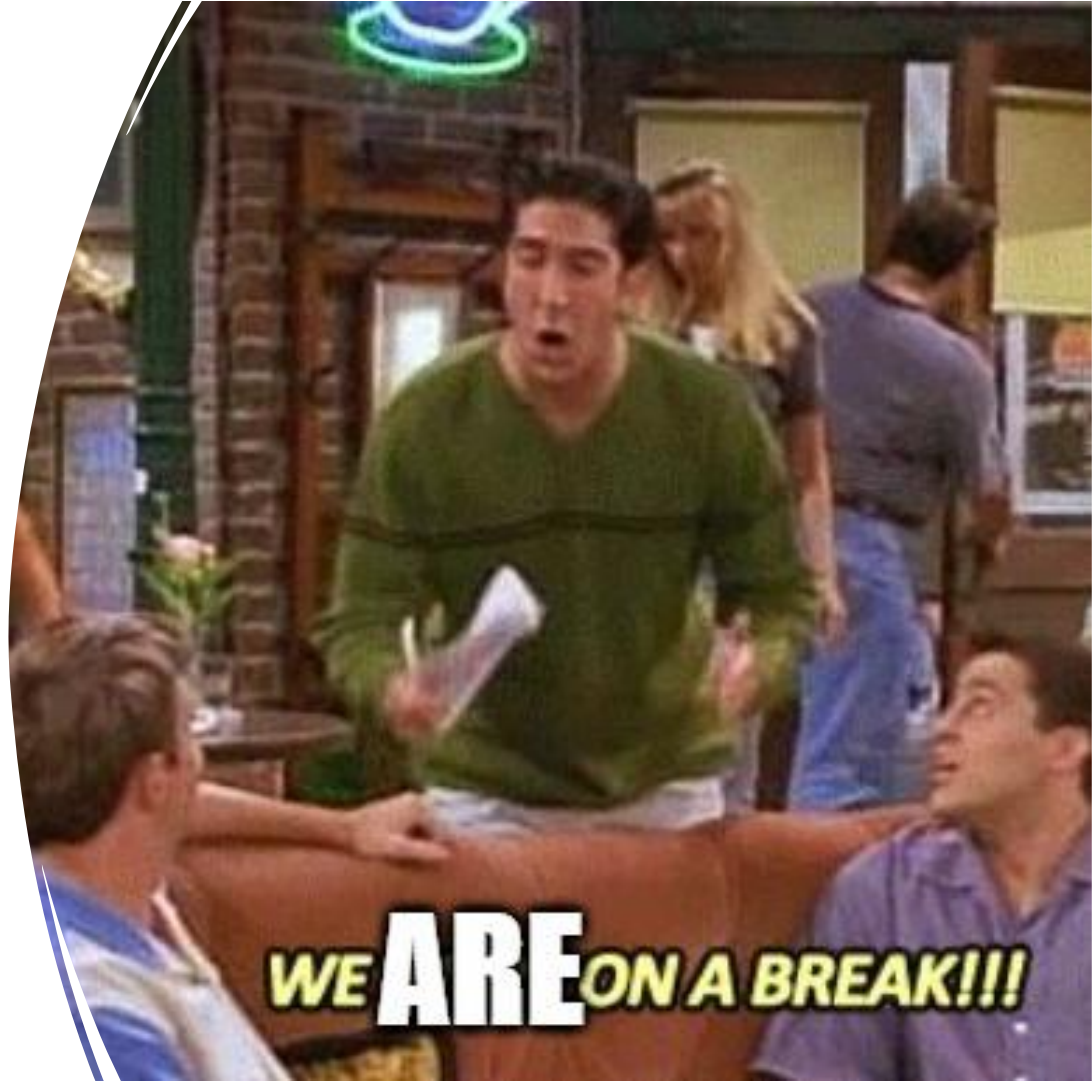
- The original B-Tree from 1972 **stored keys and values in all nodes in the tree.**
- More space-efficient, since each key only appears once in the tree.
- A B+ Tree **only stores values in leaf nodes.** Inner nodes only guide the search process.

Summary

- Overview on B-Tree Index
- How to read the node
- Each node equals to disk block
- Disk-based multi-level balanced tree index structure

Break

- Now we will take a few minutes break
- Make sure we have everything you need for the **Quiz**



B+ Trees for Database Indexes

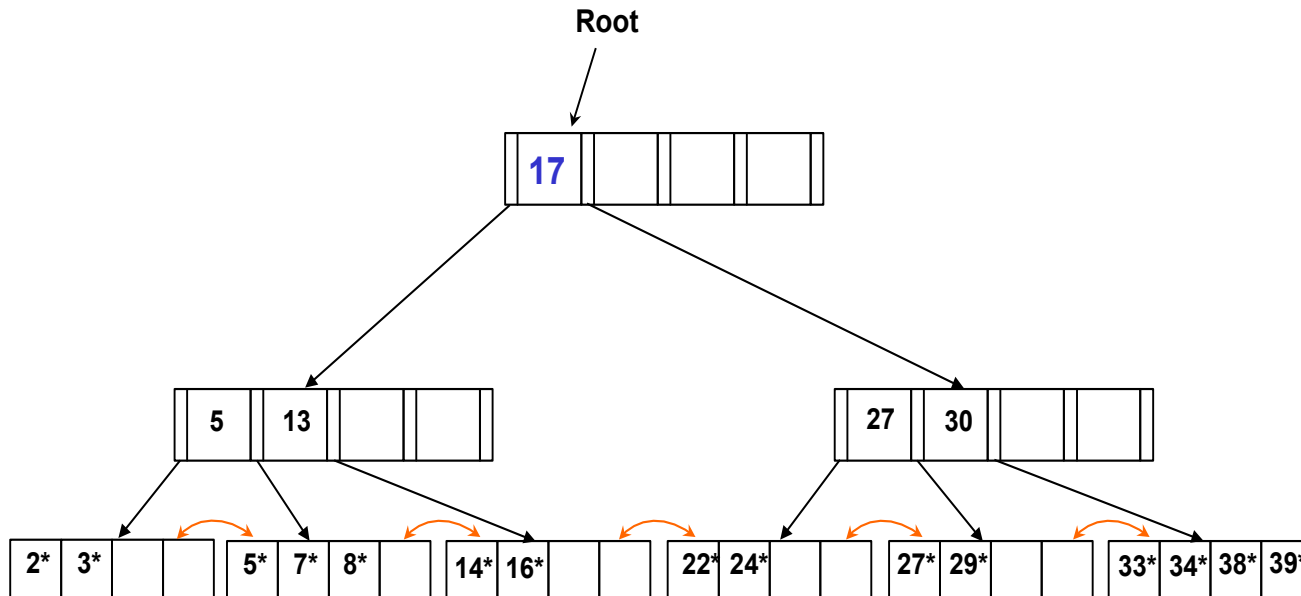


Wait....

- Dense or Sparse ?
- Primary Index or Secondary Index?

Dense vs. Sparse

- **In Theory, leaf level can be either dense or sparse**
 - Sparse only if the data file is sorted
- **In Practice, in most DBMS the leaf level is always dense**
 - One index entry for each data record
- **What about values in non-leaf levels?**
 - **Not necessarily** a subset of leaf values



B-Tree in Practice

- Assume a disk block of 4k bytes = 4096
- Assume indexing integer keys (4 bytes) & each pointer is 8 bytes
- How many entries N can fit in one B-tree node
 - $4N + 8(N+1) = 4096 \quad \rightarrow \quad N = 340$
- **A 3-Level B-Tree (root + 1st and 2nd levels) can index how many records**
(Maximum # of keys)

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

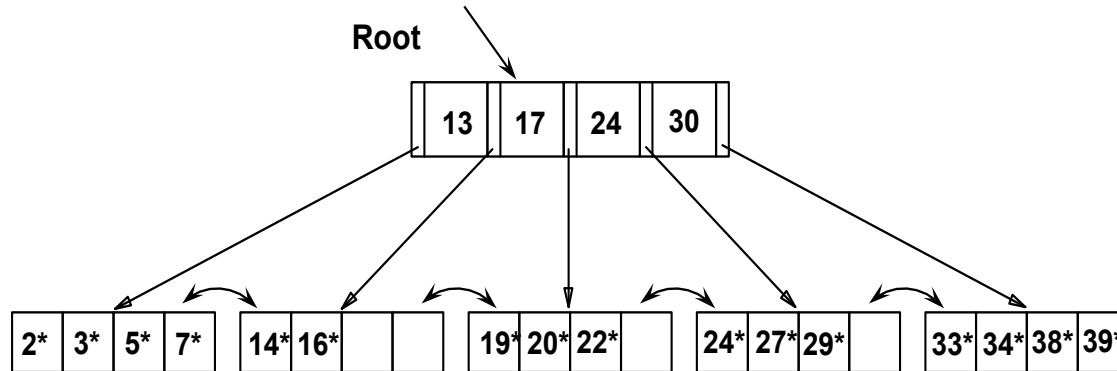
- First level (root) $\rightarrow$ 1 Node [340 keys, 341 pointers]
- 2nd level $\rightarrow$ 341 Nodes [each one contains 340 keys, 341 pointers]
- 3rd level $\rightarrow 341^2 = 116281$ Nodes {leaf nodes, each has 340 keys}
- So max number of indexed records is = **116281 x 340 = 39,535, 540 records**

Very efficient & Compact
Structure

B-Tree in Practice

- Assume a disk block of 4k bytes = 4096
 - Assume indexing integer keys (4 bytes) & each pointer is 8 bytes
 - How many entries N can fit in one B-tree node
 - $4N + 8(N+1) = 4096 \rightarrow N = 340$
 - **A 3-Level B-Tree (root + 1st and 2nd levels) can index how many records (On Average)**
 - First level (root) $\rightarrow$ 1 Node [Max=341, min= 2, avg=**171** pointers]
 - 2nd level $\rightarrow$ 1 x **171** Nodes [Max=341, min= 171, avg = **256**]
 - 3rd level $\rightarrow$ **171** x **256** = **43776** Nodes {leaf nodes, each has **255** on average}
 - So Avg number of indexed records is = **43776** * **255** = **11,162,880 records**
- Use at least
Non-leaf: $\lceil (n+1)/2 \rceil$ pointers
Leaf: $\lfloor (n+1)/2 \rfloor$ pointers to data

B-Tree in Practice



- Usually 3- or 4-level B-tree index is enough to index even large files
- **The height of the tree is usually very small**
 - Good for searching, insertion, and deletion
 - Few I/Os (especially that the root can be always cached in memory)

B-Tree Querying

Queries on B-Trees

- **Equality Search:** Find all records with a search-key value of k .

1. Start with the root node

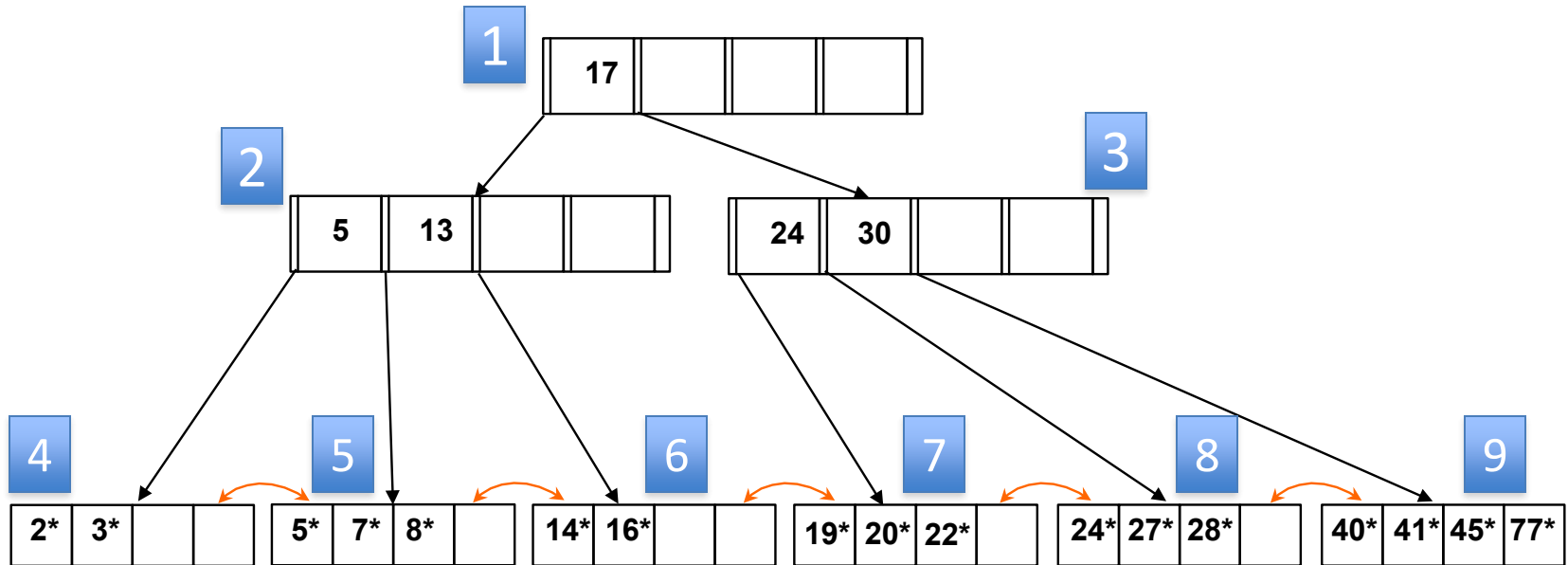
- Examine the keys (use binary search) until find a pointer to follow

2. Repeat until reach a leaf node

3. Search keys in the leaf node for the first key = k

4. Follow the pointers to the data records

B-Tree: Equality Search



Find key = 0, 2, 28, 101 ?

0 → N1, N2, N4

2 → N1, N2, N4, **then read the data block**

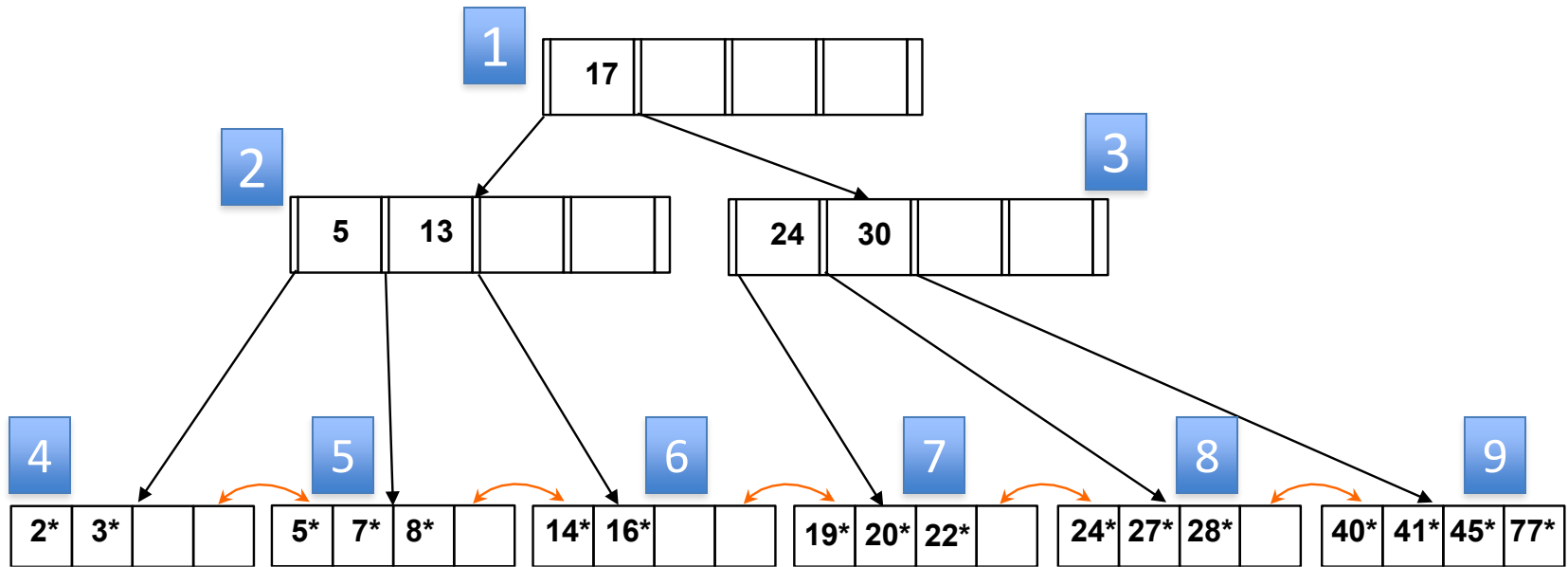
28 → N1, N3, N8, **then read the data block**

101 → N1, N3, N9

Queries on B-Trees

- **Range Search:** Find all records between $[k1, k2]$.
 1. **Start with the root node and search for $k1$**
 - Examine the keys (use binary search) until find a pointer to follow
 2. **Repeat until reach a leaf node**
 3. **Search the keys in the leaf node for the first key = $k1$ or the first larger**
 4. **Move right while the key is not larger than $k2$ (may move from one node to another node)**
 5. **Follow the pointers to the data records**

B-Tree: Range Search



Find key in [5, 20], [4, 16], [100, 200]?

[5, 20] → N1, N2, N5, N6, N7

[4, 16] → N1, N2, N4, N5, N6

[100, 200] → N1, N3, N9

Summary

- Calculate the #keys a B-tree node can hold
- Calculate the #keys that can be indexed in a B-tree of height $H \rightarrow$ MAX, AVG, MIN
- How to search over B-Tree
 - Equality
 - Range

Quiz 3

- **Reminders:**
 - **Closed book**
 - **Individual work**
 - **You have 20 minutes**

